CONFIDENTIAL

Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA

UNIVERSITI TEKNOLOGI MALAYSIA
FINAL EXAMINATION SEMESTER II, SESSION 2024/2025

PAPER 1 (THEORY)

SUBJECT CODE : SECJ1023

SUBJECT NAME : PROGRAMMING TECHNIQUE II

SECTION : 01 / 02 / 03 / 04 / 05 / 06 / 07

TIME : (2 HOURS)

DATE/DAY :

VENUES :

DO NOT OPEN THIS BOOKLET UNTIL YOU ARE TOLD TO DO SO

INSTRUCTIONS:

1. Answer ALL questions in the given answer booklet.
2. This question paper consists of TWO (2) sections.
   Section A: TWENTY (20) multiple-choice questions.
   Section B: THREE (3) structured questions.
3. Submit both the question paper and answer booklet at the end of the examination.

(Please Write Your Name and Section in Your Answer Booklet)

Name	AKMAL RAFIQUE BIN AHMAD RAPHAIE
IC/ISID No.	
Year / Course	1 / SBCPH
Section	
Lecturer Name	

WARNING: Disciplinary action will be taken against students who are found cheating during the examination.

*This question paper consists of sixteen (**16**) printed pages including this page.*

SECTION A: MULTIPLE-CHOICE QUESTIONS [30 MARKS]

Instructions:

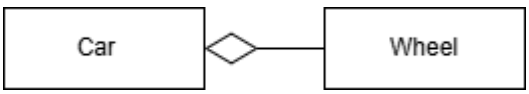
*This section consists of **twenty (20)** questions. Answer all questions in the Answer Booklet.*

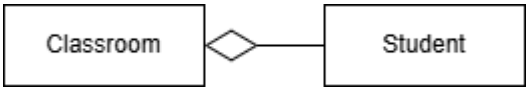
- *Read each question carefully.*
- *Choose the **BEST** answer from the options A, B, C, or D.*
- *Each question carries **1.5 marks**.*

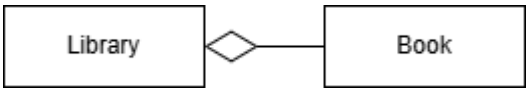
1. When a class contains an instance of another class, it is known as _____.
   A. aggregation
   B. composition
   C. inheritance
   D. polymorphism

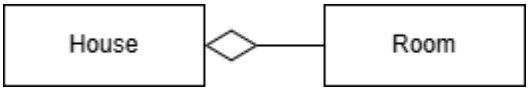
2. Which of the following scenarios **best** illustrates an aggregation relationship in object-oriented design?
   A. A `Company` object and an `Employee` object, where employees can work for other companies or exist independently.
   B. A `Document` object and a `WordProcessor` object, where the document cannot exist without the word processor.
   C. A `House` object and a `Room` object, where rooms are automatically destroyed when the house is destroyed.
   D. A `Tree` object and a `Root` object, where destroying the tree also destroys the root.

3. Which of the following diagrams shows the **incorrect** implementation of aggregation?

   A.
   

   B.
   

   C.
   

   D.
   

4. Which of the C++ code snippet below is derived from the implementation of composition?

A.
```
class Car {
public:
    void drive() { cout << "Driving car."; } };

class Driver {
public:
    void use(Car& car) { car.drive(); } };
```

B.
```
class Engine {
public:
    string type;
    Engine(string t) : type(t) {} };

class Car {
    Engine* engine;
    string model;
public:
    Car(Engine* e, string m) : engine(e), model(m) {}
    void drive() { cout << model << " is moving"; } };
```

C.
```
class Heart {
public:
    void beat() { cout << "Heart is beating."; } };

class Human {
    Heart heart;
public:
    void live() { heart.beat(); } };
```

D.
```
class Wheel {
public:
    string position;
    Wheel(string pos) : position(pos) {}
    void rotate() { cout << " wheel rotating"; } };

class Car {
    Wheel *frontLeft, *frontRight;
public:
    Car(Wheel *fl, Wheel *fr) {
        frontLeft = fl;
        frontRight = fr;
    }
    void drive() {
        frontLeft->rotate();
        frontRight->rotate();
    } };
```

5. Given the C++ code snippet below, what type of relationship is implemented?

```
1  class Pen {
2  public:
3      void write() { cout << "Writer uses a pen."; }
4  };
5
6  class Writer {
7  public:
8      void use(Pen& pen) { pen.write(); }
9  };
10
11 int main() {
12     Pen pen;
13     Writer writer;
14     writer.use(pen);
15     return 0;
16 }
```

A. Aggregation
B. Association → One class use or interact with another class but does not own it.
C. Composition
D. No relationship

6. When a derived class has two or more base classes, the situation is called _____.

A. encapsulation
B. multiple inheritance
C. multiplicity
D. polymorphism

7. The ____(i)____ constructor is called before the ____(ii)____ constructor.

Select your answer based on the order: (i) , (ii)

A. base, derived
B. derived, base
C. private, public
D. public, private

8. What will happen if Line 32: **`cout << pub.protectedVar;`** is present in the following C++ code snippet?

```
 1  class Base {
 2  public:
 3      int publicVar = 1;
 4  protected:
 5      int protectedVar = 2;
 6  private:
 7      int privateVar = 3;
 8  };
 9
10  class PublicDerived : public Base {
11  public:
12      void show() {
13          cout << "From PublicDerived: ";
14          cout << publicVar << " ";
15          cout << protectedVar << " ";
16      }
17  };
18
19  class PrivateDerived : private Base {
20  public:
21      void show() {
22          cout << "From PrivateDerived: ";
23          cout << publicVar << " ";
24          cout << protectedVar << " ";
25      }
26  };
27
28  int main() {
29      PublicDerived pub;
30      pub.show();
31      cout << pub.publicVar << endl;
32      cout << pub.protectedVar;
33      return 0;
34  }
```

A. `2`

B. `Error: 'protectedVar' is a protected member of 'Base'`

C. `From PublicDerived: 1 2`

D. `From PublicDerived: 2`

9. In the context of inheritance in C++, how does private inheritance affect the accessibility of base class members in the derived class?

   A. All members of the base class, including private, protected, and public, become public in the derived class.

   B. Only the public members of the base class are inherited and become private in the derived class.

   C. The public and protected members of the base class become private in the derived class, and cannot be accessed outside the derived class.

   D. The public and protected members of the base class remain accessible as public and protected in the derived class, respectively.

10. Given the C++ code snippet below, fill in the blanks so that the `PremiumPrinter` class can use the `print()` and `scan()` functions.

```
1  class Device {
2  public:
3      void print() { cout << "Printing document..."; }
4  };
5
6  __________ {
7  public:
8      void scan() { cout << "Scanning document..."; }
9  };
10
11 __________ {
12 public:
13     void fax() {
14         cout << "Faxing document...";
15     }
16 };
```

	Line 6	Line 11
A.	`class Scanner : public Device`	`class PremiumPrinter : public Scanner`
B.	`class Scanner : public PremiumPrinter`	`class PremiumPrinter : public Device`
C.	`class PremiumPrinter : public Scanner`	`class Scanner : public Device`
D.	`class Scanner : public Device`	`class PremiumPrinter : public Device`

11. Given the C++ code snippet, which of the following should be used in Line 3 to ensure the `Cat::speak()` function is called via a parent class pointer?

```
1  class Animal {
2      public:
3          _____ { cout << "Animal sound"; }
4  };
5
6  class Cat : public Animal {
7      public:
8          void speak() { cout << "Meow!"; }
9  };
10
11 int main() {
12     Animal* pet = new Cat();
13     pet->speak();
14     return 0; }
```

A. `Animal speak()`

B. `virtual void speak()`

C. `void speak()`

D. `string speak()`

12. Given the C++ code snippet below, which of the following concepts is **NOT** demonstrated in the code?

```
1  class Print {
2  public:
3      virtual void show() { cout << "Base Print class"; }
4  };
5
6  class PrintInt : public Print {
7      int value;
8  public:
9      PrintInt(int x) { value = x; }
10     void show() { cout << "Integer: " << value; }
11 };
12
13 class PrintString : public Print {
14     string value;
15 public:
16     PrintString(string s) { value = s; }
17     void show() { cout << "String: " << value; }
18 };
```

A. Overridden Methods

B. Polymorphism

C. Redefined Method

D. Virtual Member Functions

13. Given the C++ code snippet below, what would be the output?

```
1  class Vehicle {
2  public:
3      virtual void showType() {
4          cout << "General Vehicle\n"; } };
5
6  class Car : public Vehicle {
7  public:
8      void showType() { cout << "Car Vehicle\n"; }
9      void drive() { cout << "Driving a car\n"; } };
10
11 class SportsCar : public Car {
12 public:
13     void showType() { cout << "Sports Car\n"; }
14     void drive(int speed) {
15         cout << "Driving at speed: " << speed << " km/h\n";
16     } };
17
18 int main() {
19     Vehicle *v;
20     SportsCar sc;
21     v = &sc;
22     v->showType();
23     static_cast<Car*>(v)->drive();
24     SportsCar *scPtr = &sc;
25     scPtr->drive(100);
26     return 0;
27 }
```

A.
```
Sports Car
Driving a car
Driving at speed: 100 km/h
```

B.
```
Sports Car
Driving at speed: 100 km/h
Driving a car
```

C.
```
General Vehicle
Driving a car
Driving at speed: 100 km/h
```

D. Compilation error

14. When member functions behave differently depending on which object performed the call, this is an example of _____.

A. association

B. encapsulation

C. multiplicity

D. polymorphism

15. Given the C++ code snippet below, what would be the output?

```
 1  class Base {
 2  public:
 3      Base() { cout << "Base constructor\n"; }
 4      virtual ~Base() { cout << "Base destructor\n"; } };
 5
 6  class Derived : public Base {
 7      int* array;
 8  public:
 9      Derived(){
10          array = new int[100];
11          cout << "Derived constructor\n"; }
12      ~Derived(){
13          delete[] array;
14          cout << "Derived destructor\n"; } };
15
16  int main() {
17      Base* obj = new Derived();
18      delete obj;
19      return 0;
20  }
```

A. `Derived constructor`
`Derived destructor`

B. `Base constructor`
`Base destructor`

C. `Derived constructor`
`Base constructor`
`Base destructor`
`Derived destructor`

D. `Base constructor`
`Derived constructor`
`Derived destructor`
`Base destructor`

16. Given the C++ code snippet below, which of the following statements is the **`throw`** point?

```
1  double divide(int numer, int denom) {
2      if (denom == 0)
3          throw "ERROR: Cannot divide by zero.\n";
4      else
5          return static_cast<double>(numer)/denom;
6  }
```

A. `if (denom == 0)`

B. `return static_cast<double>(numer)/denom;`

C. `throw "ERROR: Cannot divide by zero.\n";`

D. There is no `throw` point because there is no `try` block.

17. Given the C++ code snippet below, what would be the output?

```
1  vector<int> v(3, 5);
2  cout << v[1];
```

A. Compilation error

B. 0

C. 3

Ⓓ 5

18. Which of the C++ code snippets below does not implement a **valid** template prefix?

A.
```
vector<int> v;

auto x = max<double>(5, 3.14);
```

B.
```
template<class T>

T maxima(T a, T b) {
    return a > b ? a : b;
}
```

C.
```
template <class T>

T getZero() {
    return T(0);
}
```

D.
```
template <class T>

T add(T a, T b) {
    return a + b;
}
```

19. Given the C++ code snippet below, what would be the output?

```
1  try {
2      throw "Something went wrong!";
3  }
4
5  catch (const char* errorMessage) {
6      cout << "Error: " << errorMessage << endl;
7  }
8
9  cout << "Program continues after the error.\n";
```

A. Error: Something went wrong!

B. Program continues after the error.

Ⓒ Error: Something went wrong!
Program continues after the error.

D. Something went wrong!
Error: Program continues after the error.

20. Given the C++ code snippet, rewrite the loop from Line 4 to Line 6 using an iterator. Which of the following code snippets is **correct**?

```
1  vector<string> fruits = {"apple", "banana", "cherry",
2  "date"};
3
4  for (int i = 0; i < fruits.size(); i++) {
5      cout << fruits[i] << " ";
6  }
```

A.
```
for (vector::iterator it = fruits.begin();
    it != fruits.end(); it.next())
{
    cout << *it << " ";
}
```

B.
```
for (int i = 0; i < fruits.size(); i++)
{
    cout << fruits.iterator[i] << " ";
}
```

C.
```
for (auto it = fruits.begin(); it < fruits.end(); ++it)
{
    cout << *it << " ";
}
```

D.
```
for (vector::iterator it = fruits.begin();
    it != fruits.end(); ++it)
{
    cout << *it << " ";
}
```

SECTION B: STRUCTURED QUESTIONS **[70 MARKS]**

Instructions:

*This section consists of **three (3)** questions. Answer all questions in the Answer Booklet. The marks are indicated in the question.*

1. This question is about object-oriented programming and C++ STL concepts. [25 marks]

   a) Discuss the following concepts. For each concept, provide a use case or example in C++.

      i) Inheritance and "is-a" relationship. (5 marks)

      ii) Polymorphism using virtual functions. (5 marks)

      iii) Class templates. (5 marks)

   b) Explain how exception handling in C++ improves program reliability. Include an example that uses `try`, `throw`, and `catch` to handle an input or calculation error. (4 marks)

   c) Explain **TWO** differences between `vector` and `map` containers in C++. Then, explain **ONE** similarity between them? (6 marks)

i) inheritance → - derived class can inherit base class
- making the code much simpler
- is a relationship
- inherits attributes and function
- is a relationship implying that derived class is a special version of the base class

use case:
```
class Animal {
public:
void eat() {}
};
class Dog: public Animal {
public:
void sound() {}
};
```
Dog is an Animal

ii) Polymorphism → - Poly... is the act of using the same object differently
- virtual function is used in the base class and can be overriden in the derived class.
- is a dynamic binding
- Polymorphism allows the same function call behave differently depending on the object that being refered to

use case:
```
class Vehicle {
public:
virtual void move() {}
};
class Car : public Vehicle {
public:
void move() {}
};
int main() {
Vehicle* vehicles = new Car
vehicles -> move();
return 0;
}
```

iii) Class templates → allows a class to work with diffrent data type without rewriting the same code. it provides generic programming

use case: A box class that can work with diffrent data type like string and int

```
template <class T>
class Box {
private:
T value;
T size;
};
```

b) Exception handling improve readibility by seperating error-handling code from the main logic, preventing the program from crashing if an error occured

```
try {
if(denom < 0) {
throw runtime_error("error detect");
}
catch (runtime_error& e) {
cout << e.what();
};
```

c) vector → stores elements in a linear sequence
map → stores elements in key-value pairs

vector → can be access using index
map → can acess using a unique key

similarity → both are C++ STL container that use to store collection of data and can grow/manage their size

2. This question is about object-oriented programming implementation in C++ for UML class diagram and output tracing. [20 Marks]

Given the C++ code snippet below, answer all the following questions:

```
 1  class CPU {
 2      private:
 3          string model;
 4      public:
 5          CPU(string m) {
 6              model = m;
 7          }
 8          void process(){
 9              cout << model << " CPU processing\n";
10          }
11  };
12
13  class Computer {
14      private:
15          CPU* cpu;
16          string brand;
17      public:
18          Computer(string b, string cpuModel) {
19              brand = b;
20              cpu = new CPU(cpuModel);
21          }
22          void powerOn() const {
23              cout << brand << " computer starting up with ";
24              ________; // call process function from CPU
25          }
26          ~Computer(){
27              delete cpu;
28          }
29  };
30
31  class Laptop : public Computer {
32      private:
33          float batteryLife;
34      public:
35          Laptop(string b, string cpuModel, float life):
36           Computer(b, cpuModel) {
37              batteryLife = life;
38          }
39          void checkBattery() const {
40              cout << "Battery life: " << batteryLife
41                   << " hours\n";
42          }
43  };
44
45  int main(){
46      Computer desktop("Dell", "Intel i7");
47      ________; // call powerOn function from Computer
48
49      Laptop ultrabook("Lenovo", "AMD Ryzen 5", 8.5);
```

```
50      __________; // call powerOn function from Laptop
51      __________; // call checkBattery function from Laptop
52
53      return 0;
54  }
```

a) What are the object-oriented programming concepts implemented in this code snippet?

(2 Marks)

b) Draw a UML class diagram that represents all relationships between classes.

(11 Marks)

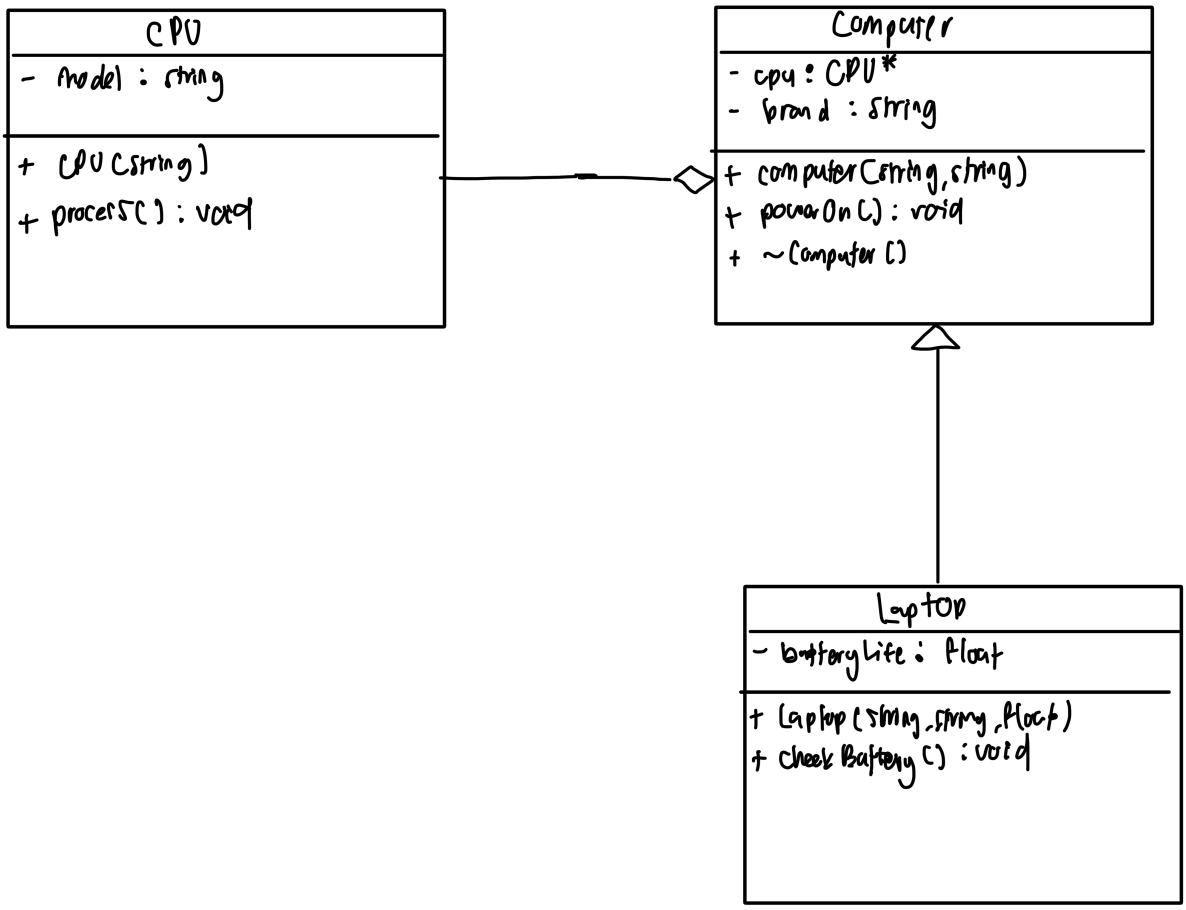
c) Fill out the missing command lines at Line 24, Line 47, Line 50, and Line 51.

(4 Marks)

d) What is the output of this program? (3 Marks)

a) inheritance and aggregation

b)



c) i) cpu -> process ()

ii) desktop.powerOn ()

iii) ultrabook.powerOn ()

iv) ultrabook.checkBattery()

d) Dell computer starting up with Intel i7 CPU processing
Lenovo computer starting up with AMD Ryzen 5 CPU processing
Battery life: 8.5 hours

3. This question is about object-oriented programming implementation in C++ for writing code segments and statements. [25 Marks]

Given the C++ code snippet below, answer all the following questions:

```
 1  class Vehicle {
 2      public:
 3          void move() { cout << "vehicle move"; }
 4  };
 5
 6  class Car : public Vehicle {
 7      public:
 8          void move() { cout << "car moving"; }
 9  };
10
11  class Boat : public Vehicle {
12      public:
13          void move() { cout << "boat sailing"; }
14  };
15
16  int main() {
17      Vehicle vehicle;
18      Car car;
19      Boat boat;
20
21      cout << "Vehicle: ";
22      vehicle.move();
23      cout << endl;
24
25      cout << "Car: ";
26      car.move();
27      cout << endl;
28
29      cout << "Boat: ";
30      boat.move();
31      cout << endl;
32
33      Vehicle* vehiclePtr;
34
35      vehiclePtr = &car;
36      cout << "Vehicle pointer to Car: ";
37      vehiclePtr->move();
38      cout << endl;
39
40      vehiclePtr = &boat;
41      cout << "Vehicle pointer to Boat: ";
42      vehiclePtr->move();
43      cout << endl;
44
45      return 0;
46  }
```

a) vehicle: vehicle move
Car: car moving
Boat: boat sailing
vehicle pointer to Car: vehicle move
vehicle pointer to boat: vehicle move

b) i) class Vehicle {
public:
virtual void move() { cout << "vehicle move"; }

virtual ~Vehicle() {};
};

c) Vehicle * myVehicle

myvehicle = new Car()
cout << "vehicle pointed to Car:";
myVehicle -> move();
cout << endl;
delete myVehicle

myvehicle = new Boat()
cout << "vehicle pointed to Boat:";
myvehicle -> move();
cout << endl;
delete myvehicle

d) vehicle: vehicle move
Car: car moving
Boat: boat sailing
vehicle pointer to Car: Car move
vehicle pointer to boat: boat move

not the same output; move() is not virtual in original code so the program used static binding, which always calls the Vehicle class move().
in new one, there is virtual move(), thus allows dynamic binding, allowing the program to check the actual object at runtime and execute the overriden move().

a) What is the output of the program? (5 Marks)

b) Rewrite the `Vehicle` class definition to support dynamic (runtime) polymorphism. To modify the code, follow these instructions:

   i) Make the `move()` function virtual in the base class (`Vehicle`) to allow the correct derived class's `move()` function to be called when using a base class pointer. (3.5 Marks)

   ii) Add a virtual destructor in the base class to ensure proper cleanup when objects are deleted through a base class pointer. (1.5 Marks)

c) Based on the modified `Vehicle` class in (b), write the C++ code that demonstrates runtime polymorphism by replacing the code from Line 33 to Line 43.

   i) Create a base class pointer (`Vehicle *`) that is assigned to derived objects (`Car` and `Boat`). (4 Marks)

   ii) Call the `move()` function to execute the correct version based on the actual object type (`Car` and `Boat`).

   Example:

   If the pointer refers to a `Car`, the output should be `"car moving"`, not `"vehicle move"`.

   If the pointer refers to a `Boat`, the output should be `"boat sailing"`, not `"vehicle move"`. (2 Marks)

   iii) Use `delete` to safely destroy the derived objects (`Car` and `Boat`). (2 Marks)

d) Based on the modified code in (b) and (c), what is the new output? Is it the same as in (a)? Explain briefly. (7 Marks)

The End…